



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Managed Vs External Tables

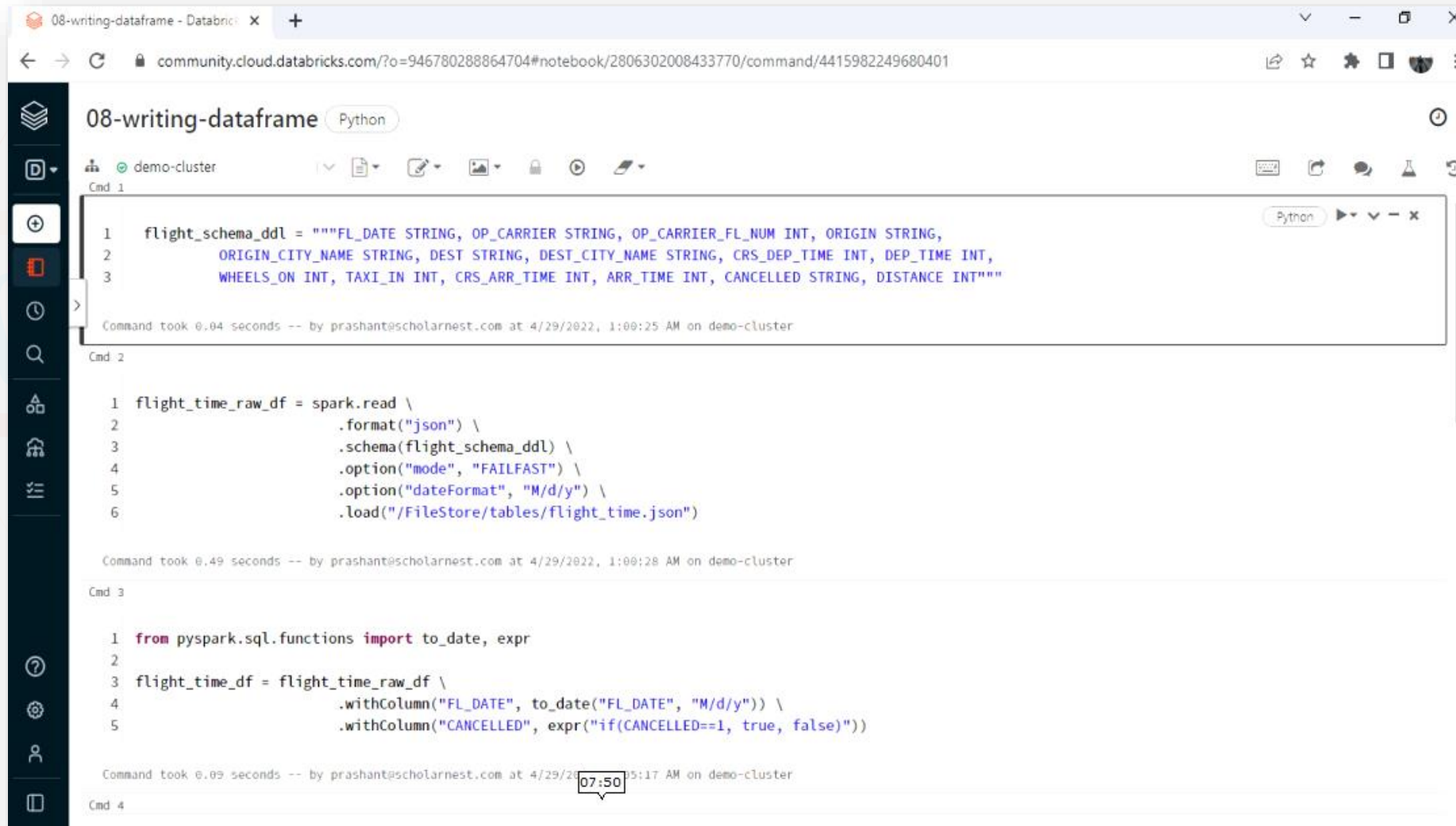
Spark allows you to create two types of tables.

1. Managed Tables

2. External Tables

Go to your databricks workspace and open an existing workbook (**Reference - 08-writing-dataframe**).

In this example, we learned to create a database table.



The screenshot shows a Databricks notebook interface with the title '08-writing-dataframe' and a 'Python' language selector. The notebook is running on a 'demo-cluster'. The left sidebar contains navigation icons. The main area displays three commands (Cmd 1, Cmd 2, Cmd 3) and their execution results.

```
Cmd 1
1 flight_schema_ddl = """FL_DATE STRING, OP_CARRIER STRING, OP_CARRIER_FL_NUM INT, ORIGIN STRING,
2   ORIGIN_CITY_NAME STRING, DEST STRING, DEST_CITY_NAME STRING, CRS_DEP_TIME INT, DEP_TIME INT,
3   WHEELS_ON INT, TAXI_IN INT, CRS_ARR_TIME INT, ARR_TIME INT, CANCELLED STRING, DISTANCE INT"""

Command took 0.04 seconds -- by prashant@scholarnest.com at 4/29/2022, 1:00:25 AM on demo-cluster

Cmd 2
1 flight_time_raw_df = spark.read \
2   .format("json") \
3   .schema(flight_schema_ddl) \
4   .option("mode", "FAILFAST") \
5   .option("dateFormat", "M/d/y") \
6   .load("/FileStore/tables/flight_time.json")

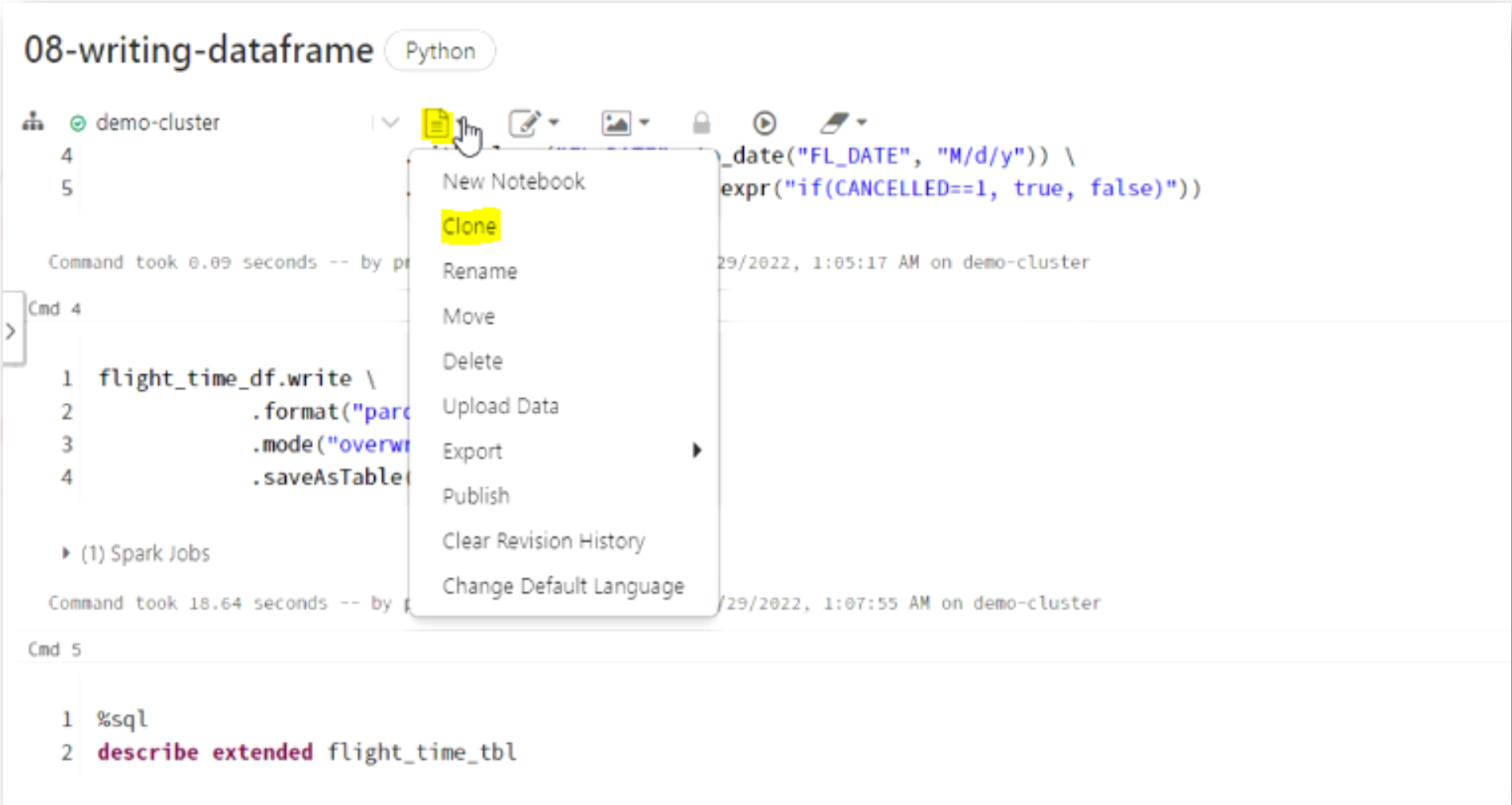
Command took 0.49 seconds -- by prashant@scholarnest.com at 4/29/2022, 1:00:28 AM on demo-cluster

Cmd 3
1 from pyspark.sql.functions import to_date, expr
2
3 flight_time_df = flight_time_raw_df \
4   .withColumn("FL_DATE", to_date("FL_DATE", "M/d/y")) \
5   .withColumn("CANCELLED", expr("if(CANCELLED==1, true, false)))

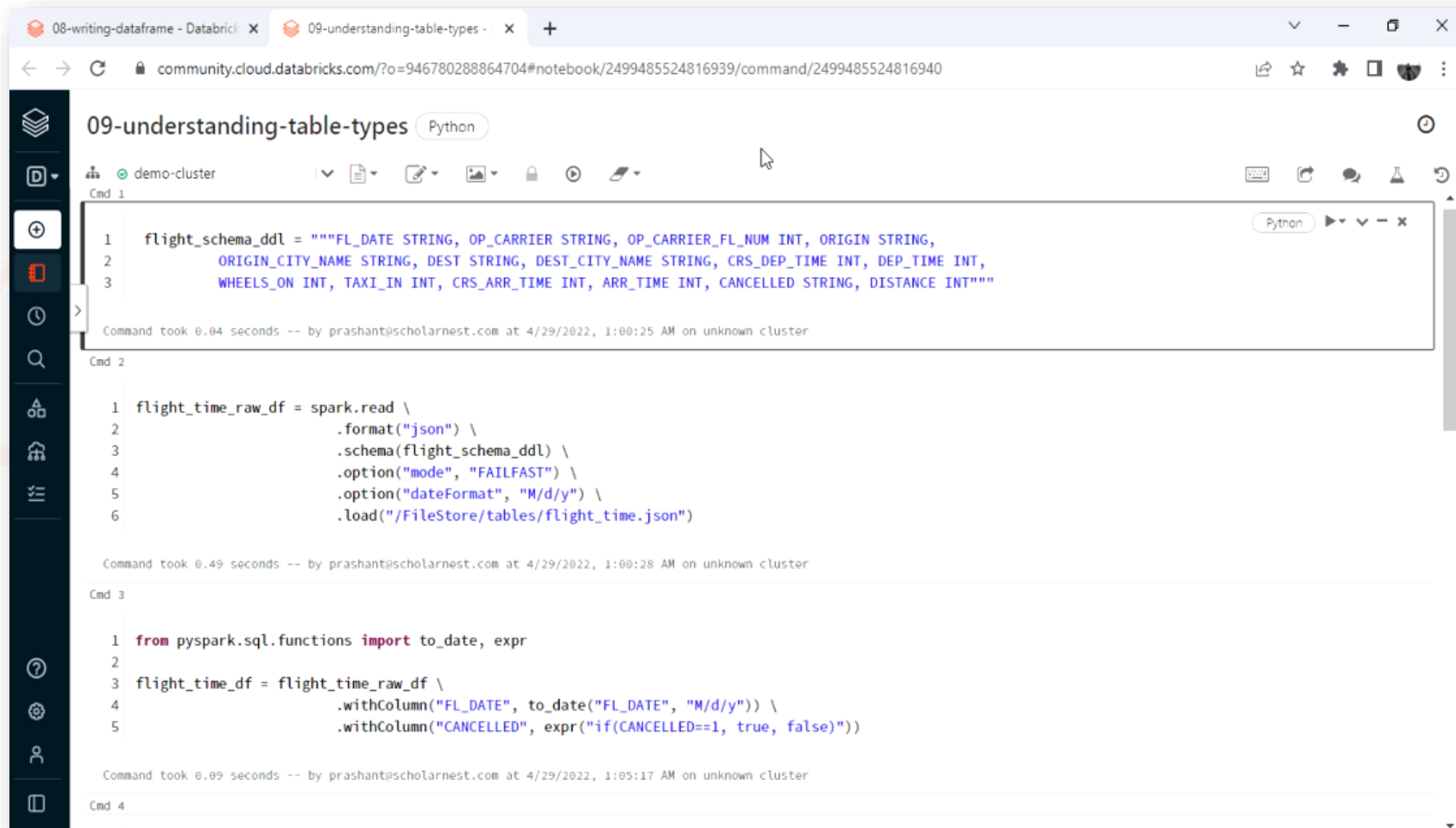
Command took 0.09 seconds -- by prashant@scholarnest.com at 4/29/2022, 1:00:35:17 AM on demo-cluster

Cmd 4
```

Click the file menu and clone your notebook as shown below.



Once cloned, rename your notebook and we will start working on this notebook.
(Reference - 09-understanding-table-types.ipynb)



The screenshot shows a Databricks notebook interface with the title '09-understanding-table-types' and a 'Python' language selector. The notebook is running on a 'demo-cluster'. The left sidebar contains navigation icons for workspace, recent notebooks, search, and other functions. The main area displays three commands (Cmd 1, Cmd 2, Cmd 3) with their respective code and execution status.

```
Cmd 1
1 flight_schema_ddl = """FL_DATE STRING, OP_CARRIER STRING, OP_CARRIER_FL_NUM INT, ORIGIN STRING,
2   ORIGIN_CITY_NAME STRING, DEST STRING, DEST_CITY_NAME STRING, CRS_DEP_TIME INT, DEP_TIME INT,
3   WHEELS_ON INT, TAXI_IN INT, CRS_ARR_TIME INT, ARR_TIME INT, CANCELLED STRING, DISTANCE INT"""

Command took 0.04 seconds -- by prashant@scholarnest.com at 4/29/2022, 1:00:25 AM on unknown cluster

Cmd 2
1 flight_time_raw_df = spark.read \
2   .format("json") \
3   .schema(flight_schema_ddl) \
4   .option("mode", "FAILFAST") \
5   .option("dateFormat", "M/d/y") \
6   .load("/FileStore/tables/flight_time.json")

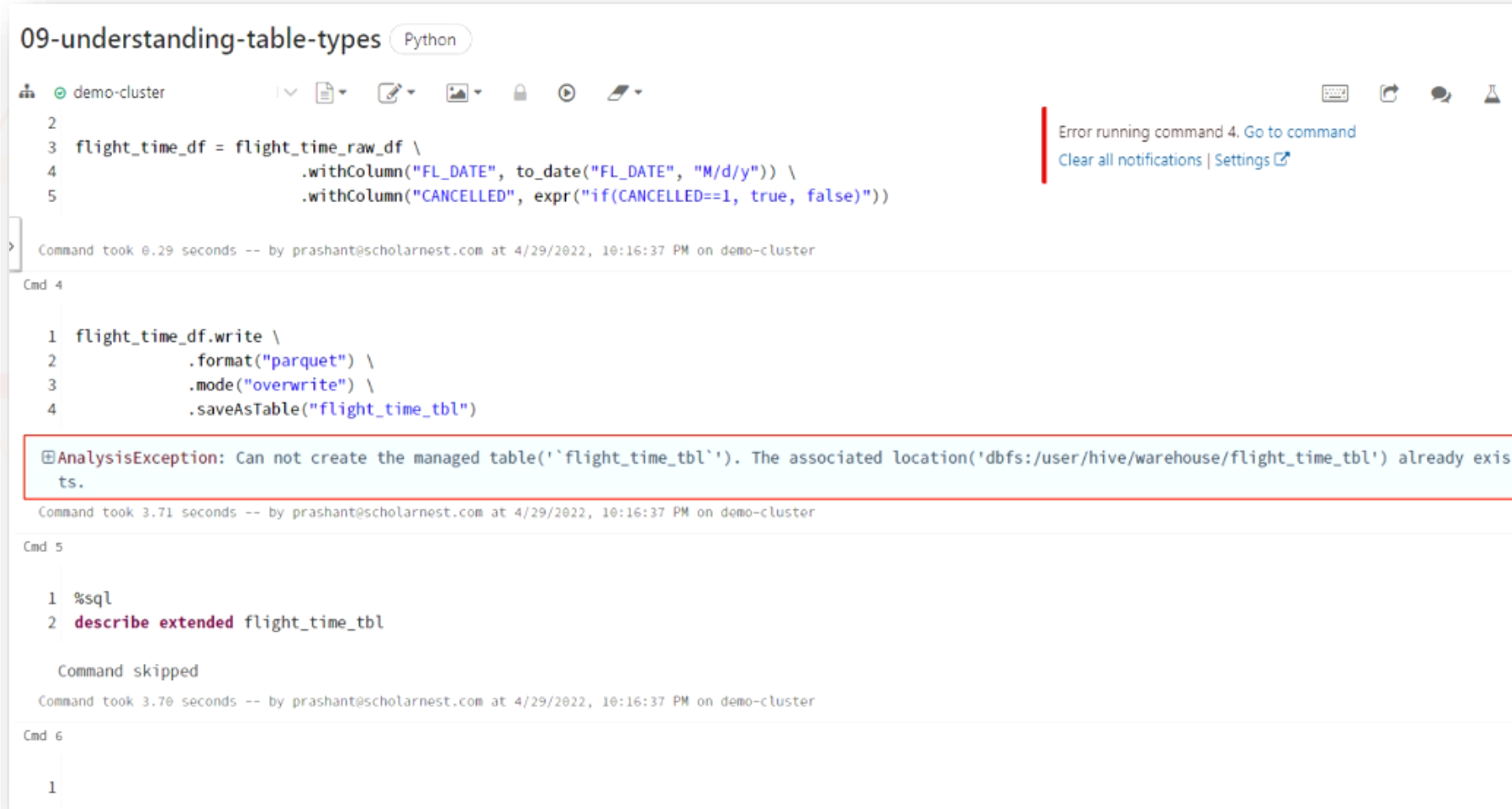
Command took 0.49 seconds -- by prashant@scholarnest.com at 4/29/2022, 1:00:28 AM on unknown cluster

Cmd 3
1 from pyspark.sql.functions import to_date, expr
2
3 flight_time_df = flight_time_raw_df \
4   .withColumn("FL_DATE", to_date("FL_DATE", "M/d/y")) \
5   .withColumn("CANCELLED", expr("if(CANCELLED=1, true, false)))

Command took 0.09 seconds -- by prashant@scholarnest.com at 4/29/2022, 1:05:17 AM on unknown cluster

Cmd 4
```

If you try running this notebook once. You might see an error as shown below. Your cluster was terminated, and you created a new cluster. So, you will see this error. Because the metadata and compute layers were removed by the Databricks. But the data files are still there.



The screenshot shows a Databricks notebook titled "09-understanding-table-types" in Python. The notebook is running on a cluster named "demo-cluster". The code defines a DataFrame `flight_time_df` and attempts to write it to a table named `flight_time_tbl` using the `write` method. The error message, highlighted in a red box, states: `AnalysisException: Can not create the managed table('flight_time_tbl'). The associated location('dbfs:/user/hive/warehouse/flight_time_tbl') already exists.` The error occurred during the execution of Command 4. Command 5, which was to describe the table, was skipped because the command was already executed.

```
09-understanding-table-types Python
demo-cluster
2
3 flight_time_df = flight_time_raw_df \
4     .withColumn("FL_DATE", to_date("FL_DATE", "M/d/y")) \
5     .withColumn("CANCELLED", expr("if(CANCELLED==1, true, false)))

Command took 0.29 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:16:37 PM on demo-cluster

Cmd 4

1 flight_time_df.write \
2     .format("parquet") \
3     .mode("overwrite") \
4     .saveAsTable("flight_time_tbl")

AnalysisException: Can not create the managed table('flight_time_tbl'). The associated location('dbfs:/user/hive/warehouse/flight_time_tbl') already exists.

Command took 3.71 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:16:37 PM on demo-cluster

Cmd 5

1 %sql
2 describe extended flight_time_tbl

Command skipped

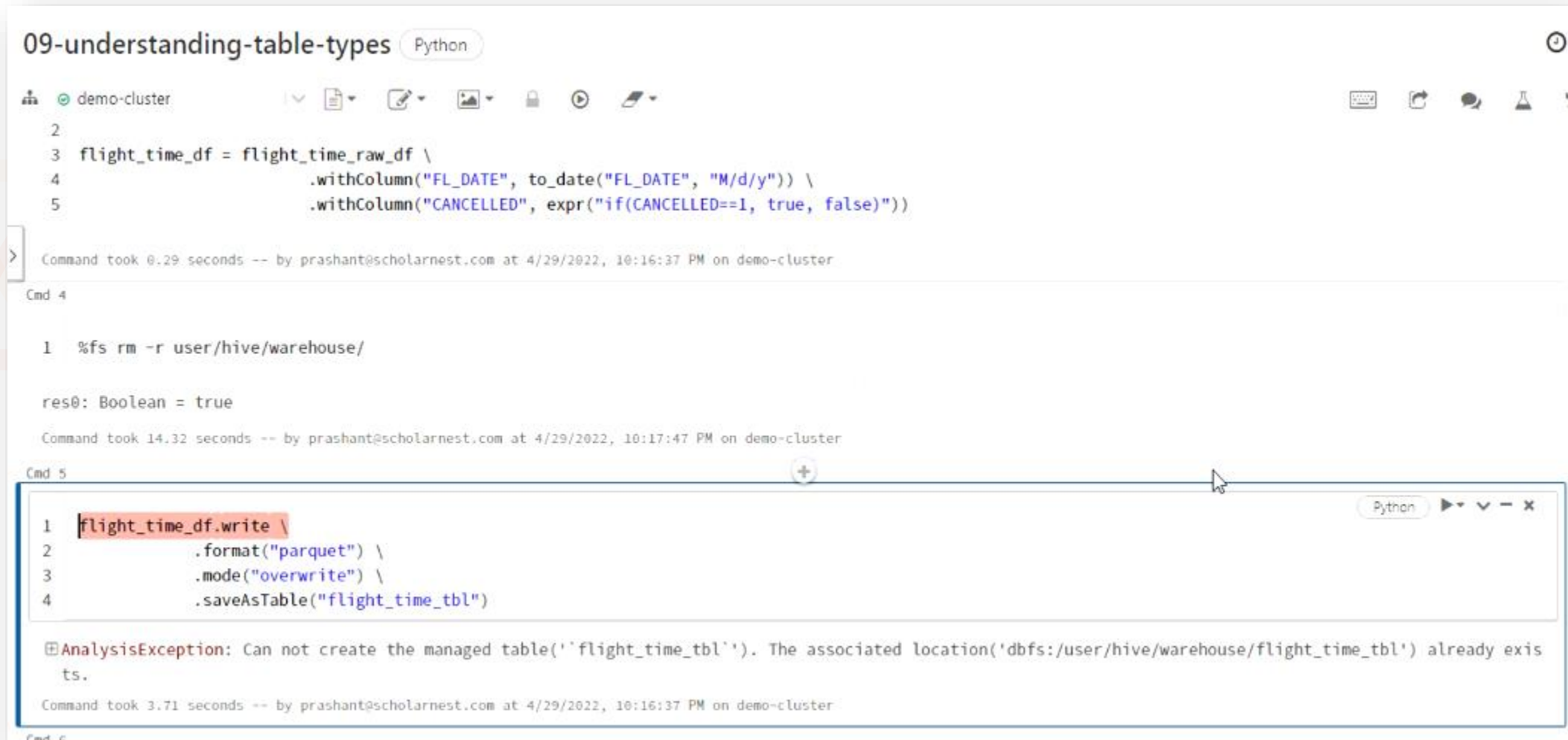
Command took 3.70 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:16:37 PM on demo-cluster

Cmd 6

1
```

You can fix the error using the %fs rm -r command, add a new cell above and write the %fs command as shown below.

Now, if you try to run the cells it will work perfectly fine.



09-understanding-table-types Python

demo-cluster

```
2
3 flight_time_df = flight_time_raw_df \
4     .withColumn("FL_DATE", to_date("FL_DATE", "M/d/y")) \
5     .withColumn("CANCELLED", expr("if(CANCELLED=1, true, false)))
```

Command took 0.29 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:16:37 PM on demo-cluster

Cmd 4

```
1 %fs rm -r user/hive/warehouse/
```

res0: Boolean = true

Command took 14.32 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:17:47 PM on demo-cluster

Cmd 5

```
1 flight_time_df.write \
2     .format("parquet") \
3     .mode("overwrite") \
4     .saveAsTable("flight_time_tbl")
```

AnalysisException: Can not create the managed table('flight_time_tbl'). The associated location('dbfs:/user/hive/warehouse/flight_time_tbl') already exists.

Command took 3.71 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:16:37 PM on demo-cluster

Cmd 6

Scroll down to see the output of the describe command, and you will see the table type. This table is a managed table.

So by default, the `saveAsTable()` method created a managed table.

You can also create a table using the create table DDL statement.

That one also creates a managed table by default.

```
1 %sql
2 describe extended flight_time_tbl
```

	col_name	data_type	comment
23	Created By	Spark 3.2.1	
24	Type	MANAGED	
25	Provider	parquet	
26	Statistics	3767904 bytes	
27	Location	dbfs:/user/hive/warehouse/flight_time_tbl	
28	Serde Library	org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe	

Showing all 30 rows.

Command took 6.87 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:18:15 PM on demo-cluster

We have two properties of a managed table:

1. Spark creates managed table at a predefined warehouse location.
2. Spark manages table metadata and table data together.

You might ask the following questions about the first point:

1. Can we define this location?
2. Can we change it?

The answer is YES-NO.

You can configure the default warehouse directory location using the `spark.SQL.warehouse.dir` configuration. You can go to Spark UI environment tab and search for the `spark.SQL.warehouse.dir` configuration.

The value for this configuration will be redacted for security reasons.

But the `spark.SQL.warehouse.dir` defines the default location for the managed tables.

You can set this configuration before starting the Spark cluster.

There is two things to learn here:

1. Spark creates all the managed tables at a fixed warehouse directory location.
2. You can set the warehouse directory location using the `spark.SQL.warehouse.dir` configuration.

We have two properties of a managed table:

1. Spark creates managed table at a predefined warehouse location.
2. Spark manages table metadata and table data together.

Now, moving on to the second topic.

Spark manages table metadata and table data together.

It means, Spark creates table data and metadata both when you create a managed table.

Similarly, Spark deletes the table data and metadata both when you drop the table.

We created the flight_time_tbl. So spark created data in the /user/hive/warehouse/flight_time_tbl directory. And Spark also created metadata in the metadata store. You can check the data directory using the %fs ls command as shown below. You will see the results, there are some parquet files here. These are your data files. Spark stored my data in the parquet files.

09-understanding-table-types Python

demo-cluster

1 %fs ls /user/hive/warehouse/flight_time_tbl

	path	name
1	dbfs:/user/hive/warehouse/flight_time_tbl/_SUCCESS	_SUCCESS
2	dbfs:/user/hive/warehouse/flight_time_tbl/_committed_876508015745917614	_committed_876508015745917614
3	dbfs:/user/hive/warehouse/flight_time_tbl/_started_876508015745917614	_started_876508015745917614
4	dbfs:/user/hive/warehouse/flight_time_tbl/part-00000-tid-876508015745917614-1dc261a9-121e-4d78-af8e-fc75cd79f8f4-336-1-c000.snappy.parquet	part-00000-tid-876508015745917614-1dc261a9-121e-4d78-af8e-fc75cd79f8f4-336-1-c000.snappy.parquet
5	dbfs:/user/hive/warehouse/flight_time_tbl/part-00001-tid-876508015745917614-1dc261a9-121e-4d78-af8e-fc75cd79f8f4-337-1-c000.snappy.parquet	part-00001-tid-876508015745917614-1dc261a9-121e-4d78-af8e-fc75cd79f8f4-337-1-c000.snappy.parquet
6	dbfs:/user/hive/warehouse/flight_time_tbl/part-00002-tid-876508015745917614-1dc261a9-121e-4d78-af8e-fc75cd79f8f4-338-1-c000.snappy.parquet	part-00002-tid-876508015745917614-1dc261a9-121e-4d78-af8e-fc75cd79f8f4-338-1-c000.snappy.parquet
	dbfs:/user/hive/warehouse/flight_time_tbl/part-00003-tid-876508015745917614-1dc261a9-121e-4d78-af8e-fc75cd79f8f4-339-1-c000.snappy.parquet	part-00003-tid-876508015745917614-1dc261a9-121e-4d78-af8e-fc75cd79f8f4-339-1-c000.snappy.parquet

Showing all 11 rows.

Command took 2.44 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:22:42 PM on demo-cluster

We have two methods to query the metadata store:

1. SQL Commands
2. Spark Catalog API

```
1 %sql
2 show tables
```

	database	tableName	isTemporary
1	default	flight_time_tbl	false

Showing all 1 rows.



Command took 0.39 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:23:27 PM on demo-cluster

Cmd 9

```
1 spark.catalog.listTables()
```

► (1) Spark Jobs

```
Out[10]: [Table(name='flight_time_tbl', database='default', description=None, tableType='MANAGED', isTemporary=False)]
```

Command took 0.82 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:24:42 PM on demo-cluster

Cmd 10

Now let me drop the table.

It was a managed table, so Spark will delete the table metadata and also delete the data files.

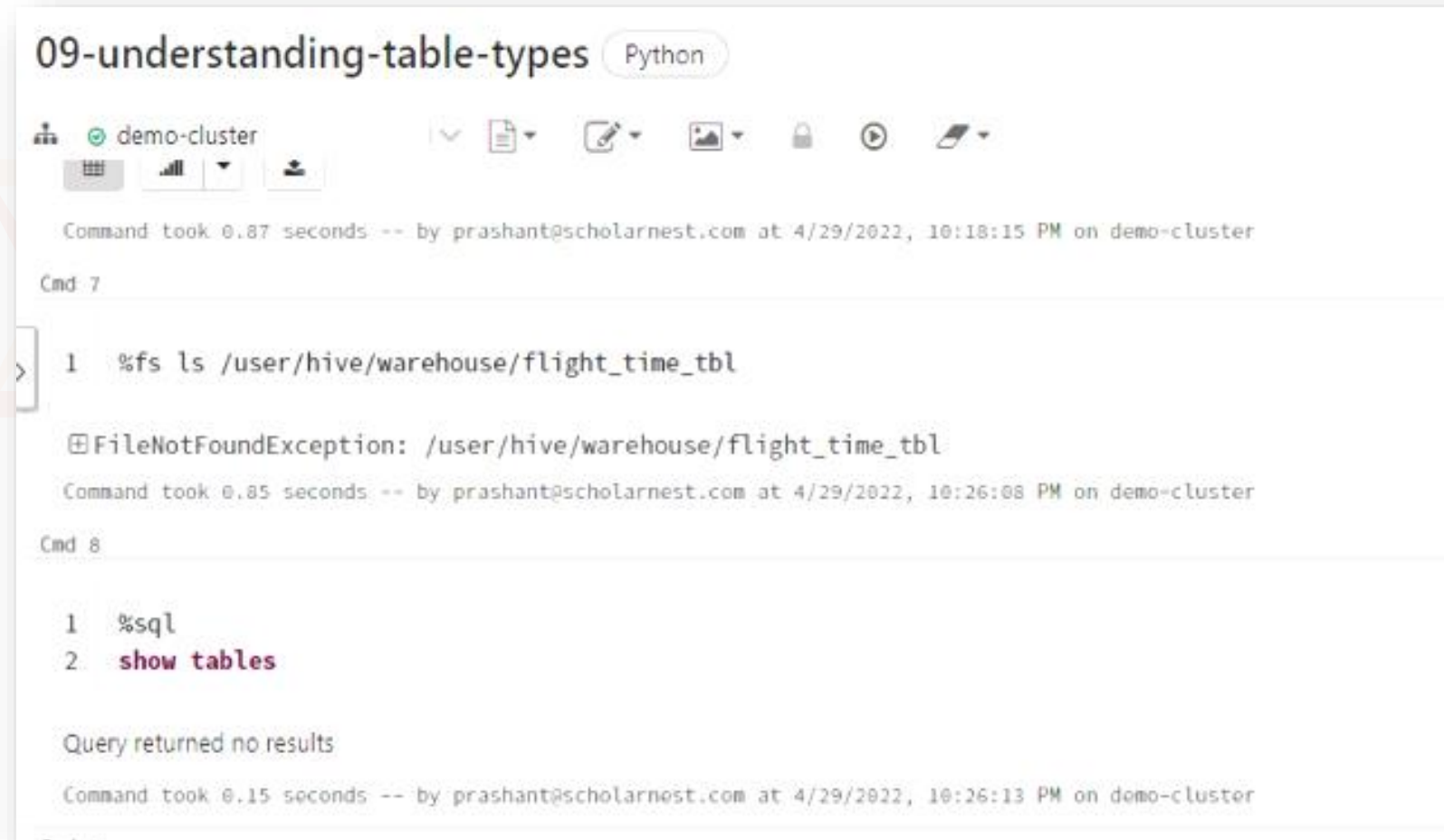
Cmd 10

```
1 %sql
2 drop table flight_time_tbl
```

OK

Command took 4.03 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:25:23 PM on demo-cluster

Now you can run your `$fs ls` command, and you will see an *FileNotFoundException*. So the table data is gone. Let me re-run the `show tables` command. And you can see the Query returned no results. So the metadata is also gone.



09-understanding-table-types Python

demo-cluster

Command took 0.87 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:18:15 PM on demo-cluster

Cmd: 7

```
1 %fs ls /user/hive/warehouse/flight_time_tbl
```

FileNotFoundException: /user/hive/warehouse/flight_time_tbl

Command took 0.85 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:26:08 PM on demo-cluster

Cmd: 8

```
1 %sql
2 show tables
```

Query returned no results

Command took 0.15 seconds -- by prashant@scholarnest.com at 4/29/2022, 10:26:13 PM on demo-cluster

Now let's talk about the external tables.

The external tables are also referred to as unmanaged tables.

However, the external table is a more popular name.

So what are external tables?

Spark offers an external table so you can create a spark table over shared external data.

External tables are a mechanism for sharing data across projects or storage layers.

Assume you have two projects. The first project collects and manages customer information. So you have a customer master table in the project. The table data is stored in some directory in the project warehouse directory location. Now you are starting a new project for retail data analysis. This project has a different warehouse directory location where you store the data.

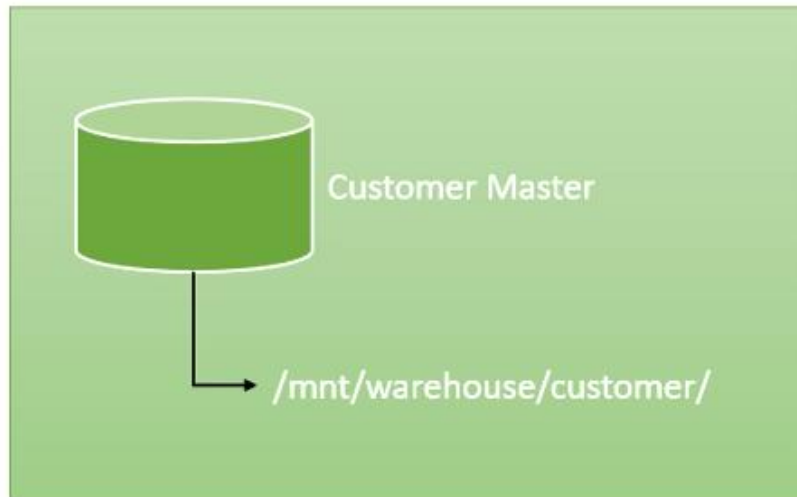
We have two separate projects and two different data stores.

Both the projects belong to the same organization, but we have two different data owners.

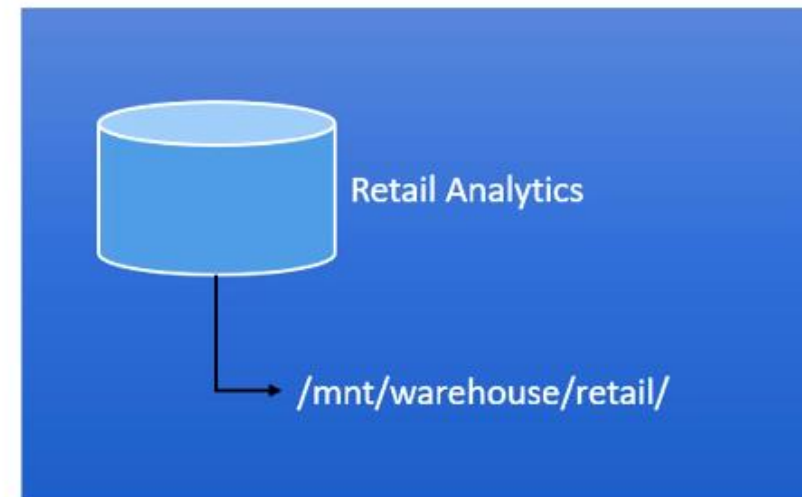
Project A owns the customer data, and project B owns the retail transactions.

But you have a requirement in Project B to access and use the customer master table.

Project A – Customer Information System



Project B – Retail Analytics System



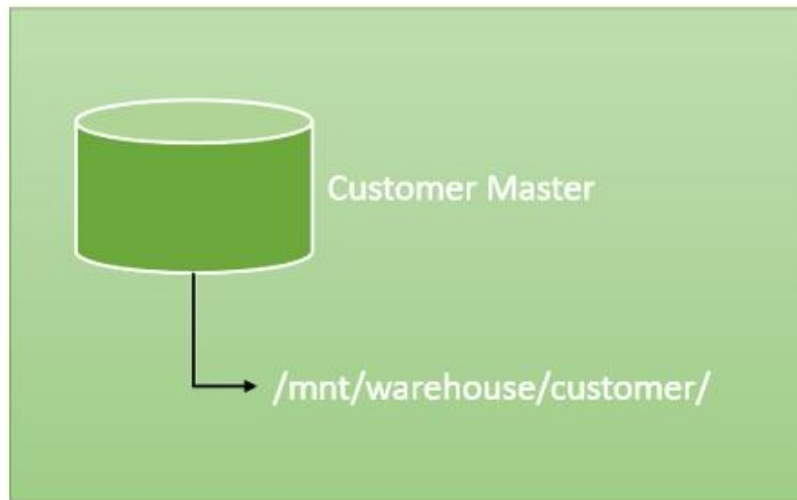
We have two approaches to solve our problem:

1. Copy the customer master data from project A to project B and use it.
2. Create an external table from project A to project B and use the same dataset.

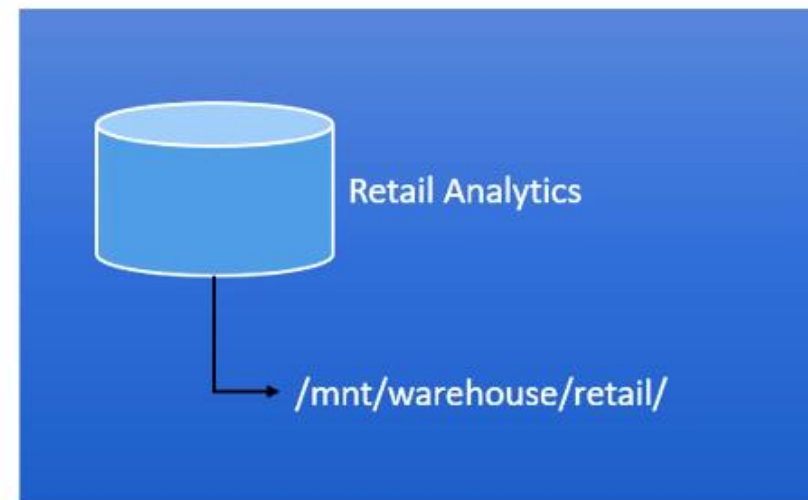
The first approach is problematic.

Copying data at two places requires extra space, and you also need to keep both copies in sync. So it is much better to use the second approach.

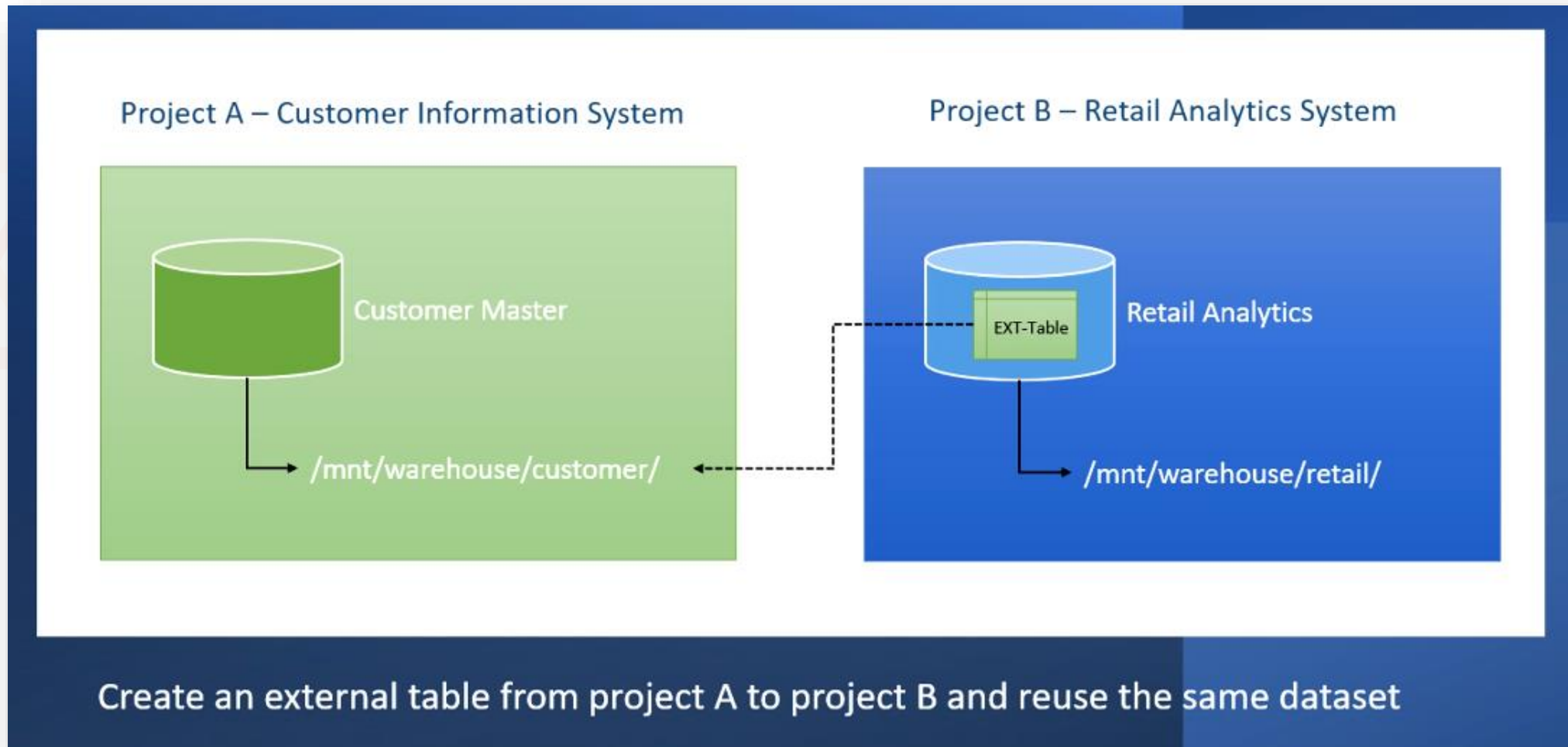
Project A – Customer Information System



Project B – Retail Analytics System



We can create an external table from project A to Project B and use the same data. So the idea behind the external table is to share data across ownerships and storage layers. Project A could be using a different storage layer than project B. But they can share the same data via external tables as long as Spark supports both storage layers.



I dropped the flight_time_tbl table, so let me recreate it.

Assume the flight_time_tbl is a Project A managed table. And Project B wants to reuse my table.

So they can create an external table on my flight_time_tbl and read data from it.

How to create an external table? They can use create table DDL as shown below.

Cmd 11

```
1 %sql
2 create table if not exists ext_flight_time_tbl like flight_time_tbl
3 location "/user/hive/warehouse/flight_time_tbl"
```

OK

Command took 0.79 seconds -- by prashant@scholarnest.com at 4/30/2022, 6:26:10 PM on demo-cluster

What is the LIKE in the statement given below? You can use the LIKE clause to copy the schema from an existing table. So it will look into the column definitions of flight_time_tbl and create the same columns and data types for ext_flight_time_tbl.

So the LIKE clause is a quick way to copy the structure of an existing table.

This statement will create a managed table. But we wanted to create an external table.

You can set the location, and spark will create an external table. We want to share the same data, so we should point this ext_flight_time_tbl to the same data files where the flight_time_tbl stores its data.

Cmd 11

```
1 %sql
2 create table if not exists ext_flight_time_tbl like flight_time_tbl
3 location "/user/hive/warehouse/flight_time_tbl"
```

OK

Command took 0.79 seconds -- by prashant@scholarnest.com at 4/30/2022, 6:26:10 PM on demo-cluster

Now you can describe it and check the table type and location.

So it is an external table, and it points to the same location. So, it means we have two tables:

- 1. flight_time_tbl
- 2. ext_flight_time_tbl

Cmd 12

1 %sql

2 describe extended ext_flight_time_tbl

	col_name	data_type	comment
21	Created Time	Sat Apr 30 12:56:11 UTC 2022	
22	Last Access	UNKNOWN	
23	Created By	Spark 3.2.1	
24	Type	EXTERNAL	
25	Provider	parquet	
26	Location	dbfs:/user/hive/warehouse/flight_time_tbl	
27	Serde Library	org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe	

Showing all 29 rows.

Command took 0.48 seconds -- by prashant@scholarnest.com at 4/30/2022, 6:26:41 PM on demo-cluster

Cmd 13

You can even check the catalog API, we have two tables:

1. flight_time_tbl
2. ext_flight_time_tbl

```
1 spark.catalog.listTables()
```

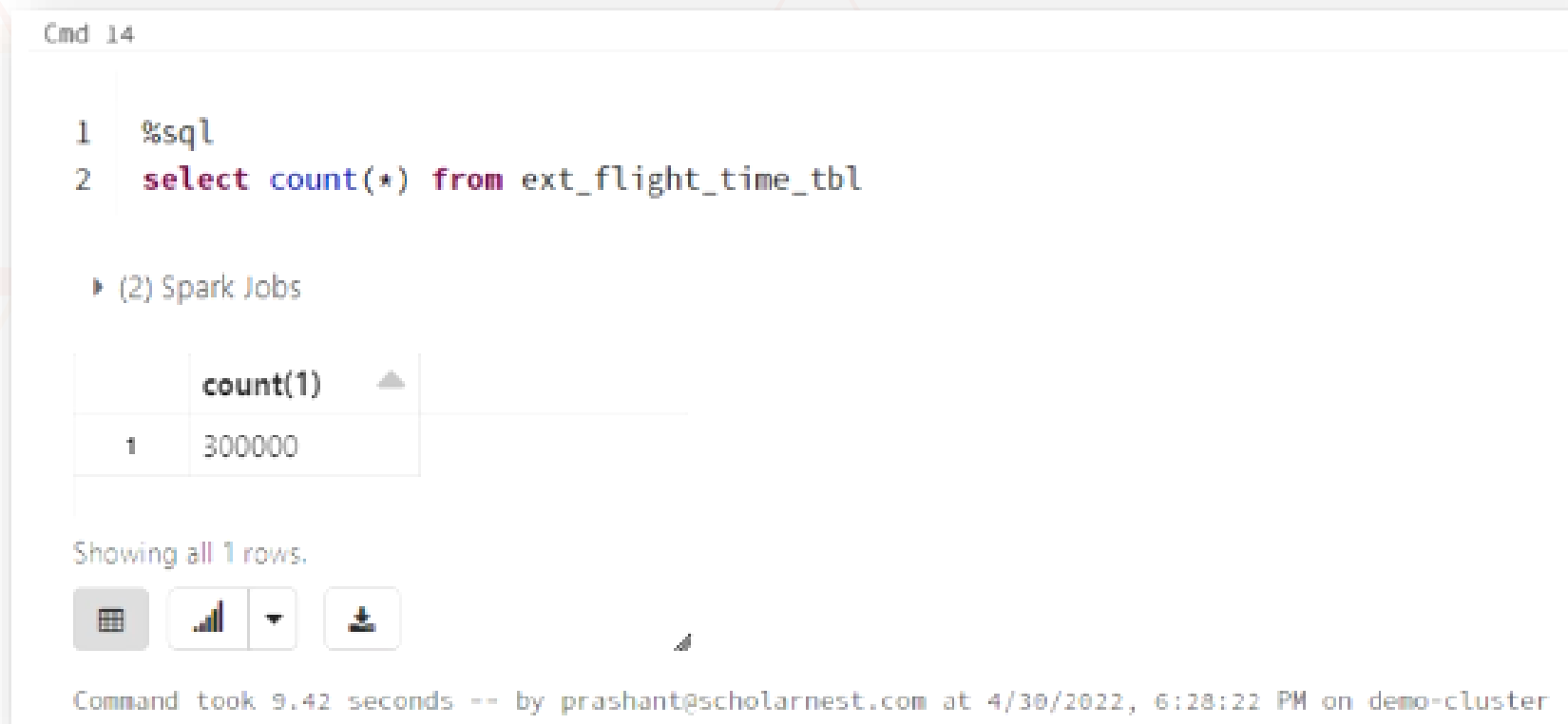
► (2) Spark Jobs

```
> Out[6]: [Table(name='ext_flight_time_tbl', database='default', description=None, tableType='EXTERNAL', isTemporary=False),  
Table(name='flight_time_tbl', database='default', description=None, tableType='MANAGED', isTemporary=False)]
```

Command took 1.12 seconds -- by prashant@scholarnest.com at 4/30/2022, 6:27:33 PM on demo-cluster

Cmd 14

You can count the records in the `ext_flight_time_tbl`. I created the table and haven't loaded the data in this external table. But you can see the record count. Because the data already exists at the given location, and the external table simply points to the same data. You can use the external table in the same way as a managed table. I mean, you can read it, update it, overwrite it or do whatever you want. But you cannot drop it. I mean, you can drop an external table, but Spark will delete the metadata only.



The screenshot shows a Databricks console window titled "Cmd 14". It contains a SQL command and its execution result. The command is: `1 %sql` followed by `2 select count(*) from ext_flight_time_tbl`. Below the command, it says "(2) Spark Jobs". The result is a table with one row and one column, showing a count of 300,000. The table has a header row with the column name "count(1)" and a data row with the value "300000". Below the table, it says "Showing all 1 rows." and there are icons for table, bar chart, dropdown, and download. At the bottom, it says "Command took 9.42 seconds -- by prashant@scholarnest.com at 4/30/2022, 6:28:22 PM on demo-cluster".

```
Cmd 14

1 %sql
2 select count(*) from ext_flight_time_tbl

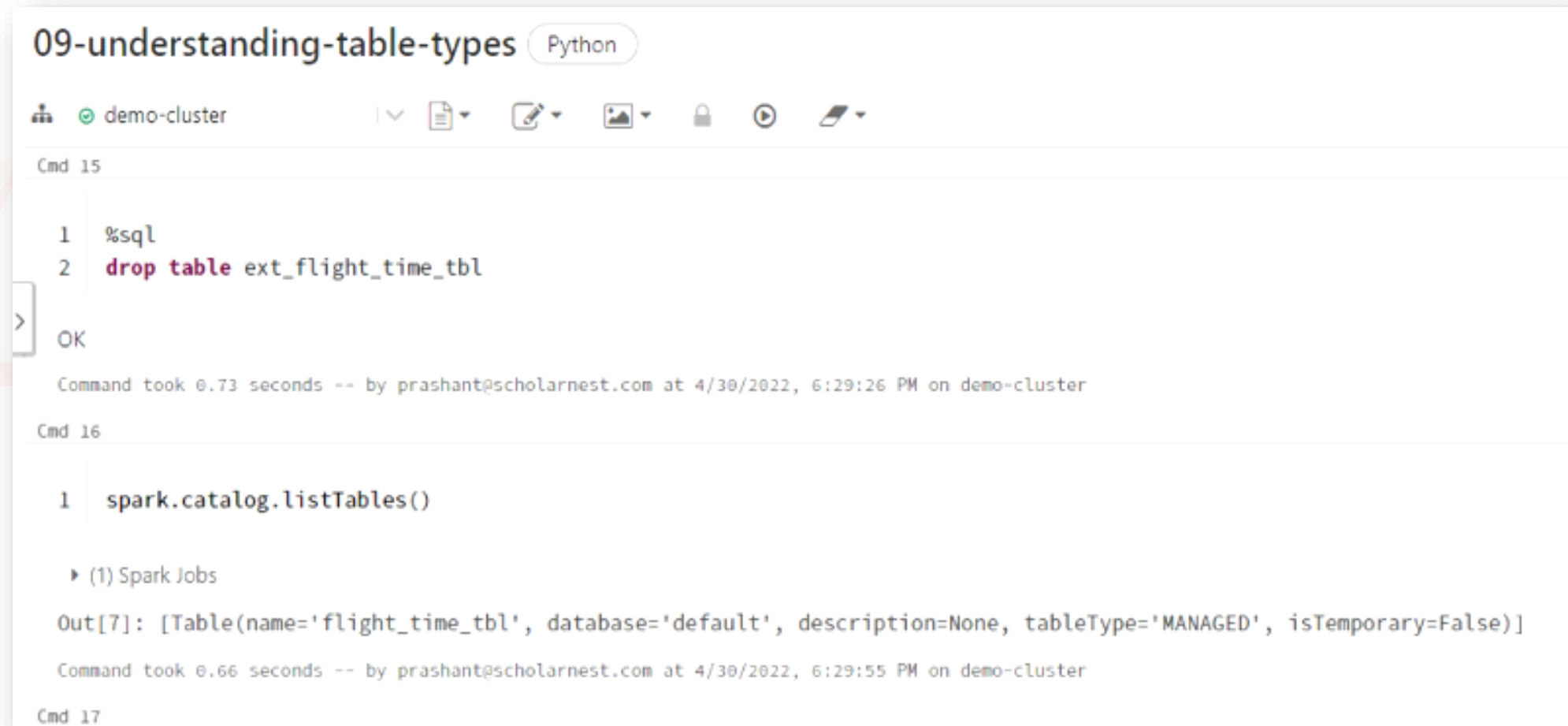
▶ (2) Spark Jobs
```

	count(1)
1	300000

Showing all 1 rows.

Command took 9.42 seconds -- by prashant@scholarnest.com at 4/30/2022, 6:28:22 PM on demo-cluster

Try the DROP statement on the external table. And now if you query the catalog, it will show you only one table.



The screenshot shows a Databricks notebook titled "09-understanding-table-types" with a "Python" language selector. The interface includes a toolbar with icons for user, cluster, file, edit, image, lock, play, and a terminal. The notebook content is divided into two command sections:

Cmd 15

```
1 %sql
2 drop table ext_flight_time_tbl
```

OK

Command took 0.73 seconds -- by prashant@scholarnest.com at 4/30/2022, 6:29:26 PM on demo-cluster

Cmd 16

```
1 spark.catalog.listTables()
```

► (1) Spark Jobs

Out[7]: [Table(name='flight_time_tbl', database='default', description=None, tableType='MANAGED', isTemporary=False)]

Command took 0.66 seconds -- by prashant@scholarnest.com at 4/30/2022, 6:29:55 PM on demo-cluster

Cmd 17

So the external table is gone. But data is still there. And it makes sense also because the original table exists. If you query the original table as shown below, you can see the records here.

```
Cmd 17

1 %sql
2 select count(*) from flight_time_tbl

▶ (2) Spark Jobs
```

	count(1)
1	300000

Showing all 1 rows.

Command took 2.50 seconds -- by prashant@scholarnest.com at 4/30/2022, 6:30:30 PM on demo-cluster

Spark Tables

1. Managed Tables

- Created at a predefined warehouse directory location
- Drop Table will delete data and metadata both

2. External Tables

- Shared data across project/storage
- Set the location/path to create an external table
- Allows all operations such as append/overwrite
- Best practice is to avoid append/overwrite external tables
- Drop table will delete metadata only



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com